

Docker: slike, kontejneri i mreže

Praktikum: Razvojne operacije · Tjedan 3 · Blok A

Sadržaj bloka

1	**Zašto Docker**	Problem okruženja, kontejner vs virtualni stroj
2	**Arhitektura i varijante**	Klijent, daemon, socket; Desktop, Engine (CLI), dind
3	**Umrežavanje**	Mreže, DNS po imenu kontejnera, veza s hostom
4	**Snimanje slike**	Dockerfile, slojevi, cache, multi-stage, korisnik, datotečni sustav, registri
5	**Kontejneri**	Slika vs kontejner, naredbe, gašenje i SIGTERM
6	**Provjera razumijevanja**	6 kratkih pitanja

1

Zašto Docker

Problem okruženja i kontejner kao rješenje

"Na mom računalu radi"

Isti kod **ne ponaša se isto** na različitim računalima jer okruženje nije dio isporuke.

Verzije

Python 3.10 kod studenta, 3.12 na poslužitelju; različit glibc, OpenSSL, sistemske biblioteke.

Ovisnosti

Dva projekta trebaju različite verzije istog paketa ili sistemske biblioteke.

Konfiguracija

Ručno postavljen poslužitelj: nitko ne zna točno što je instalirano (configuration drift).

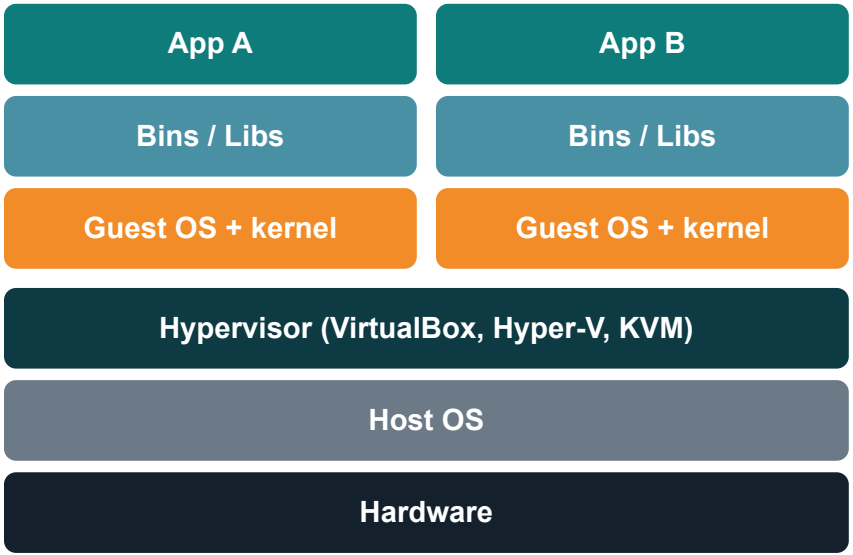
Uvođenje

Novi član tima pola dana slaže okruženje prema zastarjelom README-u.

Ideja kontejnera: aplikacija se isporučuje zajedno s okruženjem (OS biblioteke, runtime, ovisnosti, konfiguracija) kao jedna nepromjenjiva slika koja se pokreće isto na laptopu, u CI-ju i na poslužitelju.

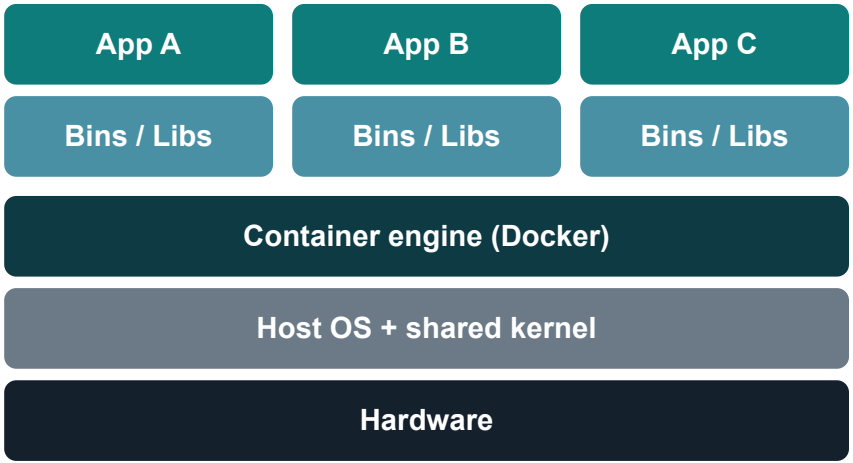
Virtualni stroj i kontejner

Virtual machines (VirtualBox)



Svaki VM ima vlastiti kernel: jača izolacija, minute i GB.

Containers (Docker)



Kontejneri dijele kernel hosta: sekunde i MB.

Kako kontejner radi: tri značajke Linux kernela

1 Namespaces

Što proces vidi

Vlastiti PID-ovi (aplikacija je PID 1), mrežna sučelja, hostname, mount točke, korisnici.

```
pid  net  mnt  uts  ipc  user
```

2 cgroups

Koliko proces smije potrošiti

Ograničenja CPU-a, memorije i I/O-a po kontejneru.

```
--memory 512m --cpus 1
```

3 Union filesystem

Iz čega se sastoji disk

Slojevi slike samo za čitanje + tanki sloj za pisanje (overlayfs).

```
docker history <image>
```

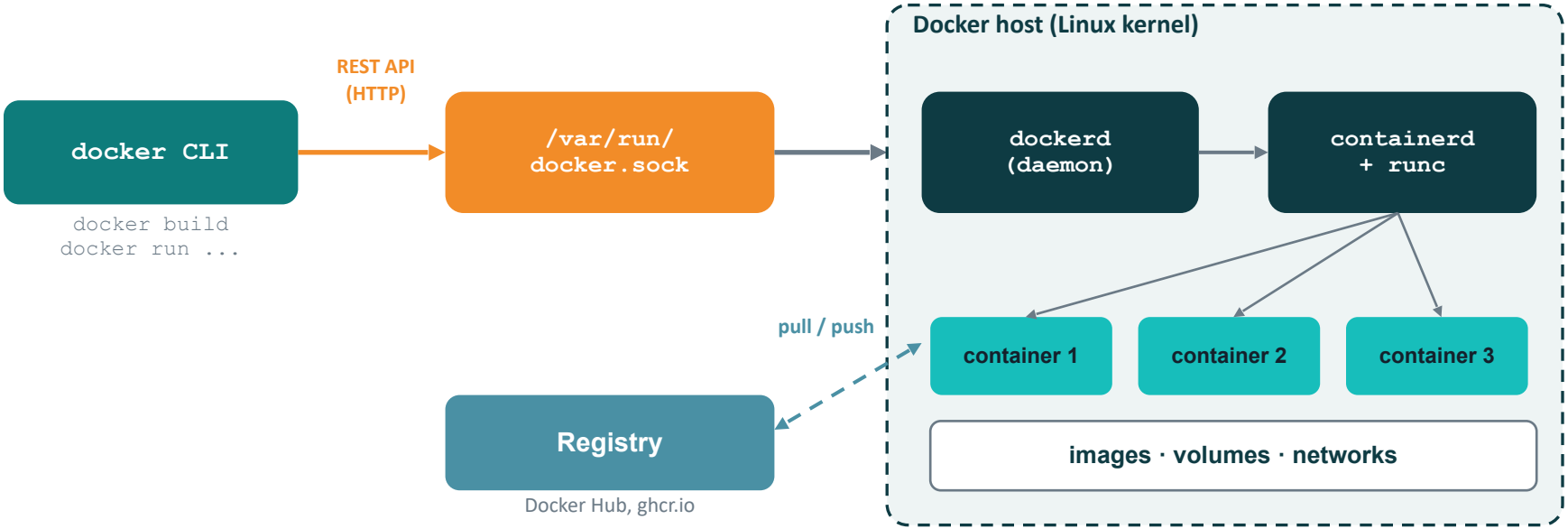
Kontejner = običan Linux proces s ograničenim pogledom i resursima. Provjera na hostu: ps aux | grep <naredba>.

2

Arhitektura i varijante

Klijent, daemon, socket; Desktop, Engine, Docker-in-Docker

Klijent, daemon i socket



CLI ne radi ništa sam: svaku naredbu šalje daemonu kroz socket. Daemon gradi slike, pokreće kontejnere i razgovara s registrom. Daemon radi kao root.

Socket: jedina vrata prema daemonu

```
# the CLI is just an HTTP client
curl --unix-socket /var/run/docker.sock \
  http://localhost/containers/json

# which daemon am I talking to?
docker context ls
echo $DOCKER_HOST     # e.g. unix:///var/run/docker.sock
                      # or tcp://build-server:2376
```

`/var/run/docker.sock`

Unix socket; vlasnik root, grupa docker. Tko može pisati u socket, upravlja daemonom.

DOCKER_HOST, docker context

Isti CLI može upravljati lokalnim ili udaljenim daemonom (SSH, TCP s TLS-om).

DevSecOps: pristup socketu = root na hostu. docker run -v /:/host ... vidi cijeli disk. Zato je grupa docker privilegija, a montiranje socketa u kontejner (-v /var/run/docker.sock:...) svjesna odluka.

Tri varijante: gdje radi daemon?

Docker Desktop

Windows, macOS

- Daemon u malom Linux VM-u
- GUI, Compose, Kubernetes
- Socket se prosljeđuje hostu
- host.docker.internal = host
- Licenca za veće tvrtke

Docker Engine (CLI)

Linux, WSL, VM u cloudu

- dockerd izravno na hostu
- Bez GUI-ja, samo docker CLI
- /var/run/docker.sock na hostu
- Kontejneri su procesi hosta
- Isto kao na poslužitelju

Docker-in-Docker (dind)

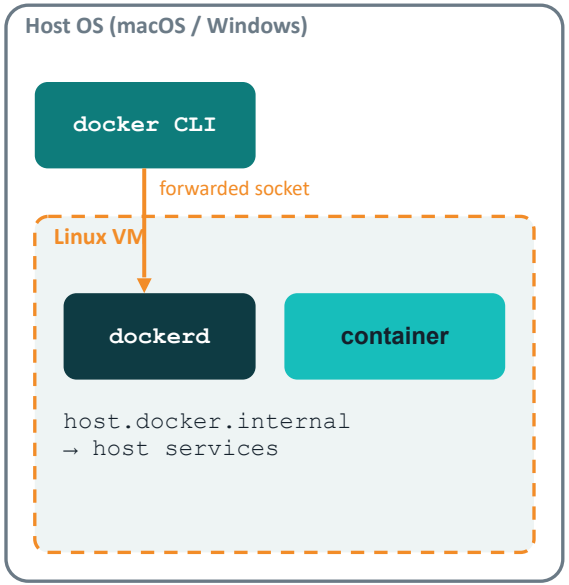
CI runneri, izolirani buildovi

- Daemon unutar kontejnera (docker:dind)
- Traži --privileged
- Vlastiti socket i cache
- Alternativa: montirati socket hosta
- Oprez: obje opcije su root

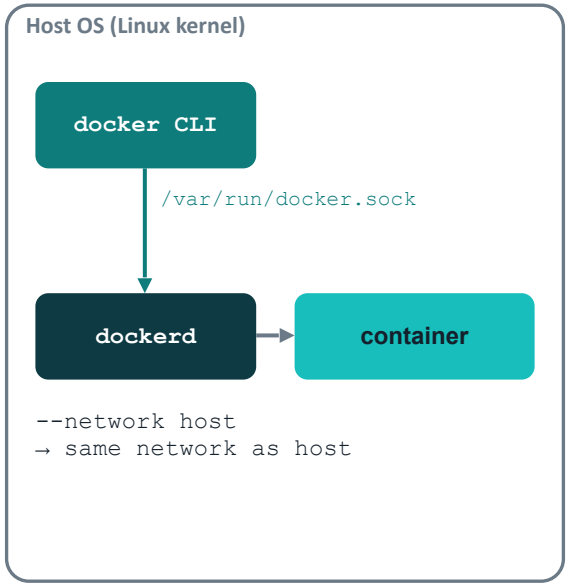
Svima isto: isti docker CLI, iste slike i isti Dockerfile; razlikuje se samo gdje radi daemon i kako kontejner vidi host.

Kako varijante razgovaraju s host OS-om

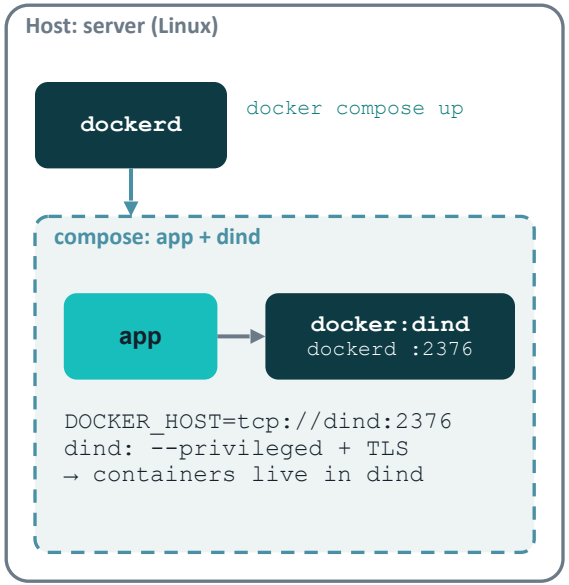
Docker Desktop



Docker Engine (Linux, WSL)



Docker-in-Docker



U ovom kolegiju: Docker Engine u WSL-u ili Docker Desktop lokalno, Docker Engine na VM-u u GCP-u.

3

Umrežavanje

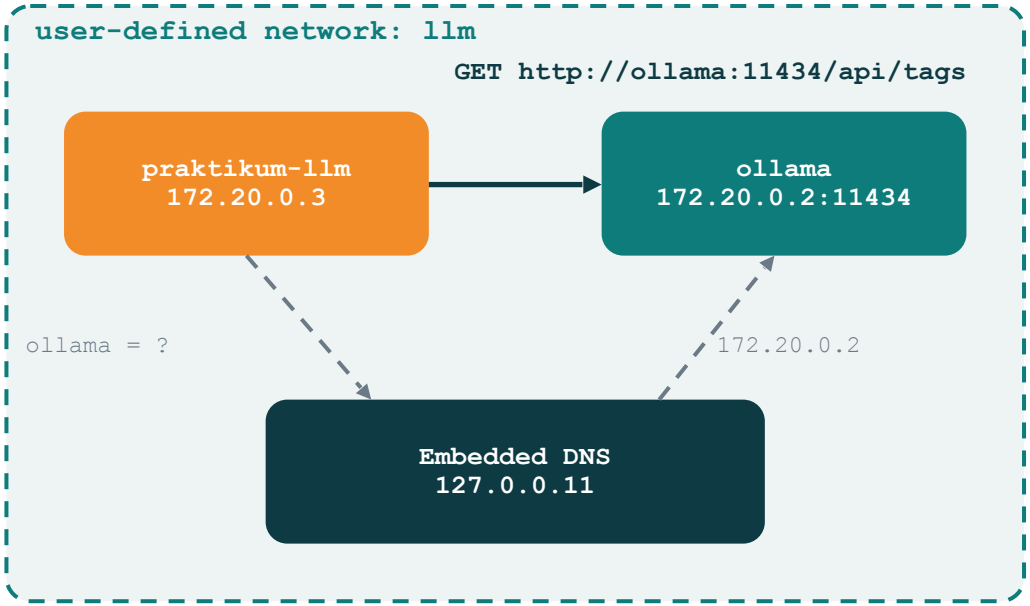
Mreže unutar Dockera, DNS po imenu, veza s hostom

Mrežni driveri

Driver	Što dobije kontejner	DNS po imenu	Kada koristiti
`bridge (default)`	Privatni IP (172.17.0.0/16), NAT prema van	Ne	Brzi test jednog kontejnera
`bridge (user-defined)`	Privatni IP na vlastitoj mreži	Da	Više kontejnera koji razgovaraju (zadano u Compose)
`host`	Mrežu hosta, bez izolacije i bez -p	Kao host	Pristup servisu na hostu (Linux)
`none`	Samo loopback	Ne	Izolirani batch poslovi
`overlay, macvlan`	Mreža preko više hostova / vlastiti MAC	Da	Swarm, posebni slučajevi

```
docker network ls
docker network create llm
docker network inspect llm     # subnet, gateway, connected containers
```

DNS: kontejneri se zovu po imenu



Pravilo: http://<container-name>:<container-port>. IP adrese se mijenjaju pri svakom pokretanju, ime ostaje. Na zadanoj bridge mreži DNS po imenu ne radi.

4

Snimanje slike

Dockerfile, slojevi, cache, multi-stage, korisnik, registri

Dockerfile: recept za sliku

```
FROM python:3.13-alpine

WORKDIR /app
ENV PYTHONUNBUFFERED=1

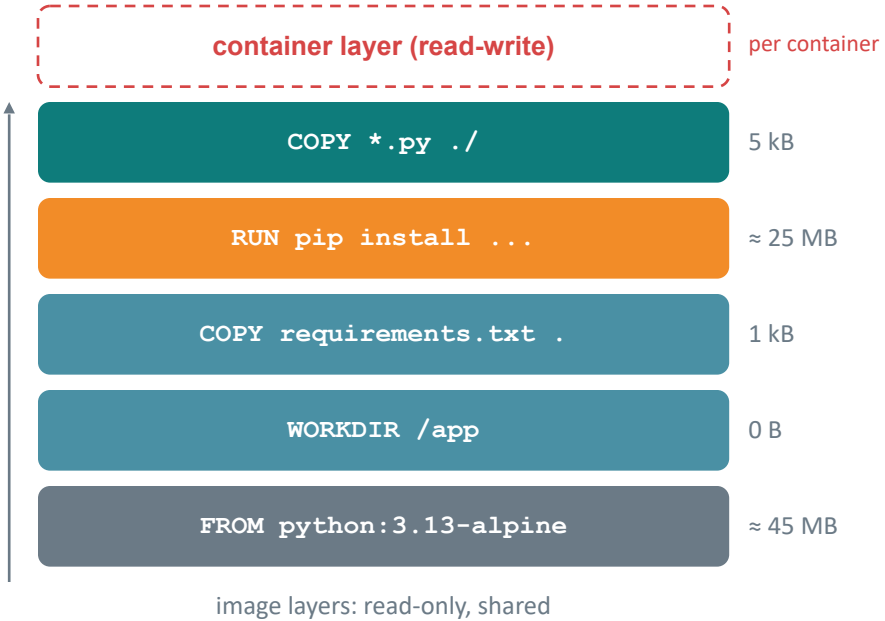
COPY requirements.txt .
RUN pip install --no-cache-dir \
    -r requirements.txt

COPY ollama_client.py llm.py ./

USER 10001
ENTRYPOINT ["python", "llm.py"]
CMD ["--help"]
```

Naredba	Što radi
`FROM`	Bazna slika (prvi sloj)
`WORKDIR`	Radni direktorij, stvara ga
`ENV / ARG`	Varijabla u kontejneru / samo u buildu
`COPY`	Datoteke iz build konteksta u sliku
`RUN`	Naredba u buildu, rezultat je novi sloj
`USER`	Korisnik za sve što slijedi
`EXPOSE`	Dokumentira port (ne objavljuje ga)
`ENTRYPOINT`	Program koji se pokreće
`CMD`	Zadani argumenti, zamjenjivi u run

Slojevi slike



Svaka naredba = sloj

RUN, COPY i ADD dodaju sloj s promjenama datotečnog sustava; ostale dodaju samo metapodatke.

Slojevi se dijele

Deset slika na istoj baznoj slici čuva baznu sliku jednom, na disku i u registru.

Copy-on-write

Kontejner piše u svoj tanki sloj. docker rm ga briše; za trajne podatke koristimo volumene (tjedan 4).

Brisanje ne smanjuje sliku

rm u kasnijem RUN-u samo skrije datoteku; zato se čisti u istom RUN-u.

Cache: redoslijed naredbi je važan

Loše: svaka promjena koda reinstalira ovisnosti

```
FROM python:3.13-alpine
WORKDIR /app
COPY . .                    # code changed
RUN pip install -r \
    requirements.txt        # cache miss
```

Dobro: ovisnosti iz cachea, mijenja se samo zadnji sloj

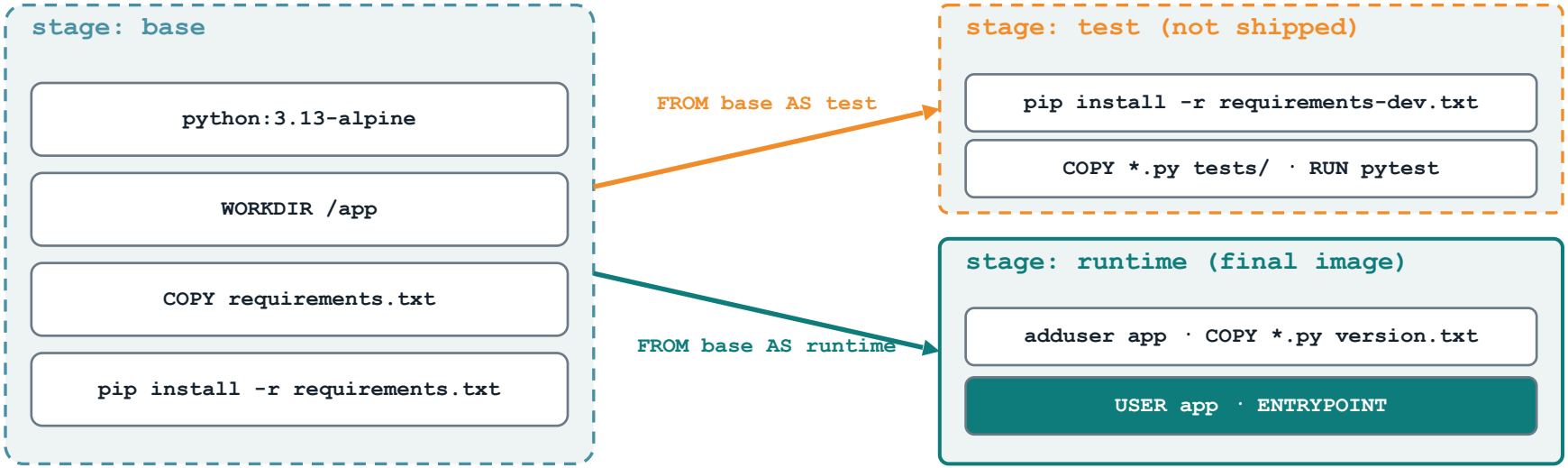
```
FROM python:3.13-alpine
WORKDIR /app
COPY requirements.txt .     # CACHED
RUN pip install -r \
    requirements.txt        # CACHED
COPY *.py ./                # rebuilt
```

Pravilo cachea: Docker ponovno koristi sloj ako su naredba i ulazne datoteke iste. Prvi promijenjeni sloj i svi iza njega grade se ponovno.

Redoslijed: ono što se rijetko mijenja (bazna slika, sistemski paketi, ovisnosti) ide gore; kod ide dolje.

Korisno: docker build --no-cache, docker build --progress plain (vidi CACHED), docker history.

Multi-stage build



Manja slika

Samo runtime ovisnosti: bez pytesta, testova i alata za razvoj.

Sigurnija slika

Manje paketa = manje ranjivosti.

Jedan Dockerfile

docker build --target test pokreće samo fazu s testovima.

Pokretanje kao ne-root korisnik

```
# alpine
RUN adduser -D -u 10001 app
# debian / ubuntu
RUN useradd --create-home -u 10001 app

USER app
WORKDIR /home/app
```

```
docker run --rm --entrypoint id img
# uid=10001(app) gid=10001(app)

docker run --rm --user 0 img ...
# override, only when really needed
```

Zašto?

Zadani korisnik u slici je root. Ako napadač probije aplikaciju ili kontejner, root u kontejneru je korak bliže rootu na hostu.

Kako?

Korisnik se stvori u slici, a USER vrijedi za sve sljedeće RUN, CMD i ENTRYPOINT. Instalacija paketa ide prije USER.

Ograničenja

Ne-root ne može slušati na portovima ispod 1024 i pisati u /usr; aplikacija piše u svoj direktorij ili volumen.

Datotečni sustav u kontejneru: /

Path	Description
`/bin`	Osnovne naredbe (sh, ls, cp)
`/boot`	Kernel i bootloader; prazno u kontejneru
`/dev`	Uređaji (null, tty, random)
`/etc`	Konfiguracija (passwd, hosts, resolv.conf)
`/home`	Direktoriji korisnika (/home/app)
`/lib`	Dijeljene biblioteke (musl ili glibc)
`/media`	Točke montiranja izmjenjivih medija
`/mnt`	Privremene točke montiranja
`/opt`	Dodatni softver izvan paketnog upravitelja

Path	Description
`/proc`	Virtualni FS: procesi i kernel
`/root`	Home direktorij roota
`/run`	Podaci o radu (PID datoteke, socketi)
`/sbin`	Sistemske naredbe (za root)
`/srv`	Podaci koje servisi poslužuju
`/sys`	Virtualni FS: uređaji i kernel
`/tmp`	Privremene datoteke
`/usr`	Programi i biblioteke (/usr/local/bin/python)
`/var`	Promjenjivi podaci: logovi, cache

Aplikacija: WORKDIR /app (kod) ili /home/app (ne-root korisnik); paket instaliran pip-om završi u /usr/local/lib/python3.13/site-packages, a naredba llm u /usr/local/bin.

.dockerignore i build kontekst

```
# .dockerignore
.git
.github
.venv
**/__pycache__
**/*.pyc
.pytest_cache
dist/
.env
```

Konfiguracija dolazi pri pokretanju: docker run -e ... ili --env-file .env

Build kontekst

docker build . šalje daemonu cijeli direktorij (točka). COPY vidi samo ono što je u kontekstu.

Brzina

Bez .venv i .git kontekst ima kB umjesto stotina MB, a promjena u .git ne poništava cache.

Tajne

.env i ključevi ne smiju završiti u slici: tko ima sliku, ima i sve slojeve (docker save, docker history).

Ponovljivost

U sliku ide samo ono što smo namjerno kopirali, ne lokalni __pycache__ ili venv za drugi OS.

Slike, tagovi i registri



Registar	Tko ga nudi
Docker Hub (docker.io)	Docker; zadani registar
GHCR (ghcr.io)	GitHub, uz repozitorij
Artifact Registry	Google Cloud (tema 6+)
ECR / ACR	AWS / Azure

Tag je pokazivač

Može se premjestiti na drugu sliku (kao git grana). latest nije "najnovija", nego zadana oznaka.

Digest je sadržaj

sha256 cijele slike; nepromjenjiv (kao git commit). Za produkciju: točna verzija ili digest.

Konvencija

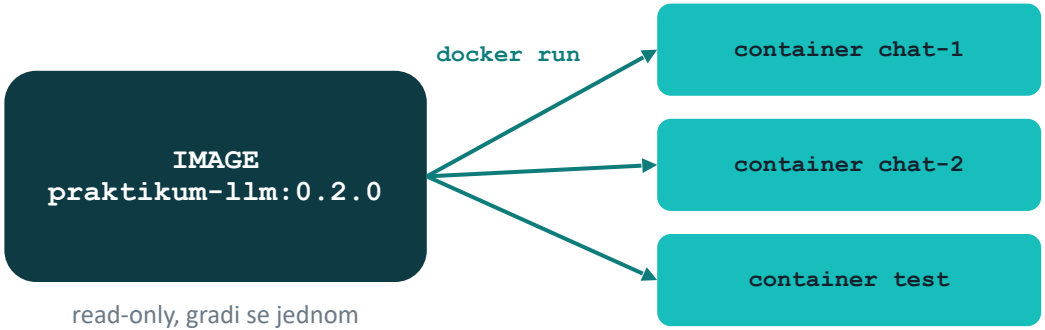
Tag = verzija iz version.txt (release-please) + kratki git SHA za CI buildove.

5

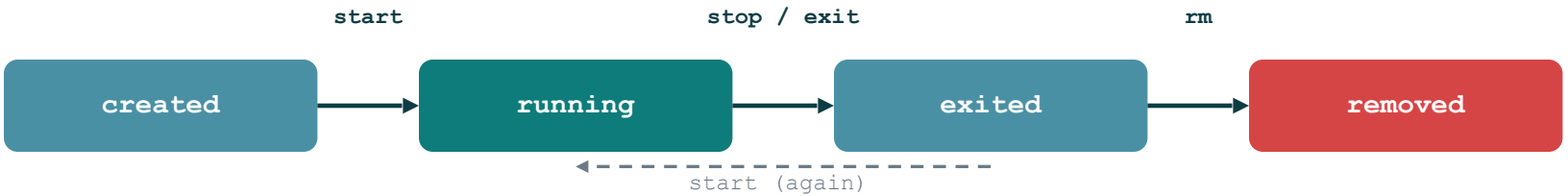
Kontejneri

Slika vs kontejner, naredbe, gašenje i SIGTERM

Slika i kontejner



Slika	Kontejner
Klasa	Objekt
Predložak	Proces koji radi
Samo za čitanje	Sloj za pisanje
Ima tag	Ima ime i ID
docker images	docker ps -a



docker run = create + start. S --rm kontejner se sam briše nakon izlaska.

Osnovne naredbe

Naredba	Primjer	Što radi
`build`	`docker build -t praktikum-llm:0.2.0 .`	Gradi sliku iz Dockerfilea i konteksta (.)
`run`	`docker run --rm -it praktikum-llm chat`	Stvara i pokreće kontejner iz slike
`exec`	`docker exec -it chat-1 sh`	Pokreće dodatni proces u kontejneru koji već radi
`stop`	`docker stop chat-1`	SIGTERM, nakon 10 s SIGKILL
`rm`	`docker rm chat-1  (docker rm -f)`	Briše zaustavljeni kontejner i njegov sloj za pisanje
`tag`	`docker tag praktikum-llm ghcr.io/org/llm:0.2.0`	Dodaje novo ime postojećoj slici (bez kopiranja)
`push`	`docker push ghcr.io/org/llm:0.2.0`	Šalje slojeve koji nedostaju u registar
`pull`	`docker pull ghcr.io/org/llm:0.2.0`	Preuzima sliku (run to radi sam ako slike nema)

`-it` = -i (stdin ostaje otvoren) + -t (pseudo-terminal). Bez toga nema interaktivnog chata ni shella. Ostalo: -d u pozadini, --name, -p, -e, --env-file, -v, --network, --init.

Gašenje kontejnera: SIGTERM i SIGKILL



PID 1 je poseban

Kernel PID-u 1 ne primjenjuje zadanu akciju za SIGTERM. Ako proces nema handler, signal se ignorira i čeka se SIGKILL. Python, Node i sleep ovdje padaju.

Exec ili shell oblik

ENTRYPOINT ["llm"]: llm je PID 1 i prima signal. ENTRYPOINT llm: PID 1 je sh, koji signal ne prosljeđuje.

Rješenja

Handler u kodu (signal.signal(SIGTERM, ...))

Zašto je uredno gašenje važno

Isporuke

Svaka nova verzija gasi stare kontejnere. 10 s čekanja po kontejneru usporava deploy i rollback.

Podaci

SIGKILL prekida upis u datoteku ili bazu na pola; handler stigne zatvoriti datoteke i veze.

Zahtjevi u tijeku

Web servis treba završiti započete zahtjeve i prestati primati nove (graceful shutdown).

Orkestratori

Compose, Kubernetes i Cloud Run šalju isti SIGTERM; ponašanje mora biti isto lokalno i u cloudu.

```
time docker stop chat-1                       # ~10 s without a handler, <1 s with one
docker inspect -f '{{.State.ExitCode}}' chat-1   # 137 or 143
```

Provjera razumijevanja

1. Koja je glavna razlika između kontejnera i virtualnog stroja u VirtualBoxu?
2. Što je /var/run/docker.sock i zašto je pristup njemu jednak root pristupu?
3. Zašto curl http://ollama:11434 radi na mreži llm, a na zadanoj bridge mreži ne radi?
4. Kojim redoslijedom pišemo COPY i RUN pip install da cache radi za nas?
5. Što je u kontejnerskom sloju za pisanje i što se s njim dogodi nakon docker rm?
6. Zašto docker stop traje 10 s i završi s kodom 137, i kako to popraviti?